

LAPORAN TUGAS BESAR TEORI BAHASA FORMAL DAN AUTOMATA

IF3170 Teori Bahasa Formal dan Automata

Milestone 2 : Syntax Analysis



Disusun Oleh:

Indah Novita Tangdililing (13523047)

Muhammad Fithra Rizki (13523049)

Sakti Bimasena (13523053)

Muhammad Timur Kanigara (13523055)

Kefas Kurnia Jonathan (13523113)

**SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
BANDUNG
2025**

DAFTAR ISI

DAFTAR ISI.....	2
1. Landasan Teori.....	3
1.1. Definisi dan Peran Parser.....	3
1.2. Keterkaitan dengan Lexer.....	3
1.3. Algoritma Recursive Descent.....	3
1.4. Parse Tree.....	4
2. Perancangan dan Implementasi.....	5
2.1. Penyesuaian Rules pada dfa_rules.json.....	5
2.2. Penyesuaian Compiler dengan Tambahan Parser dan Pembentukan Parser Tree.....	6
2.3. Penambahan Algoritma Syntax Analysis (Parser).....	7
3. Pengujian.....	12
3.1. Test Case Benar.....	12
3.2. Error, Kurang Semicolon.....	13
3.3. Error, Missing Syntax.....	14
3.4. Error, Tutup Kurung Tidak Sesuai.....	15
3.5. Error, Missing Structure.....	15
4. Kesimpulan dan Saran.....	17
4.1. Kesimpulan.....	17
4.2. Saran.....	17
5. Lampiran.....	18
6. Referensi.....	23

1. Landasan Teori

1.1. Definisi dan Peran Parser

Syntax Analysis atau Parser merupakan tahap kedua di dalam proses kompilasi setelah lexical analysis. Parser bertugas untuk memeriksa urutan dari token yang dihasilkan oleh lexical analyzer agar sesuai dengan grammar dari bahasa pemrograman yang dikompilasi.

Peran utama dari parser dalam proses kompilasi meliputi:

1. Validasi sintaks: Memastikan urutan token membentuk sintaks yang valid dan sesuai dengan grammar yang ada.
2. Deteksi error: Mendeteksi kesalahan sintaks (*syntax error*) dengan pesan yang informatif
3. Pembangunan struktur: Membangun representasi struktur hierarkis program dalam bentuk Parse Tree

Parser bertindak sebagai jembatan antara representasi linear (*token sequence*) dengan representasi hierarkis (parse tree) yang mencerminkan struktur gramatikal program.

1.2. Keterkaitan dengan Lexer

Parser sangat berkaitan dengan lexer dalam sebuah pipeline kompilasi:

1. Input-Output relationship: Lexer menghasilkan *sequence of tokens* sebagai output, yang mana parser menggunakan tokens tersebut sebagai input.
2. Sequential processing: Parser memroses tokens secara berurutan sesuai dengan urutan yang dihasilkan lexer.
3. Error coordination: Lexer mendeteksi *lexical errors*, yang mana sebuah karakter yang tidak dikenali, sedangkan parser mendeteksi *syntax errors*, yaitu urutan token yang tidak valid.
4. Position tracking: Parser menggunakan informasi posisi (seperti line line dan column) dari token untuk memberikan pesan error.

Contoh alur kerja:

Source Code: "program Hello; variabel a: integer;"
↓ (Lexer)
Tokens: [KEYWORD(program), IDENTIFIER(Hello), SEMICOLON(;), ...]
↓ (Parser)
Parse Tree: <program> → <program_header> → ...

1.3. Algoritma Recursive Descent

Recursive Descent adalah algoritma parsing yang menggunakan pendekatan *top-down* untuk membangun parse tree. Algoritma ini bekerja dengan membuat satu fungsi untuk setiap non-terminal dalam grammar.

Karakteristik dari Recursive Descent:

1. Top-down approach: Memulai dari start symbol, yaitu root, dan bergerak ke terminal symbols, yaitu leaves.
2. Grammar-driven: Struktur parser mengikuti struktur grammar secara langsung.
3. Recursive function calls: Setiap non-terminal direpresentasikan sebagai fungsi yang dapat memanggil fungsi lain ataupun dirinya sendiri.
4. LL(1) compatible: Bekerja optimal dengan grammar LL(1) yang tidak ambigu.

Kelebihan dari Recursive Descent:

1. Mudah diimplementasikan: Struktur kode mengikuti grammar secara natural.
2. Mudah di-debug: Setiap grammar rule punya fungsi tersendiri.
3. Error handling yang baik: Dapat memberikan pesan error yang spesifik.
4. Modular: Setiap fungsi bertanggung jawab untuk satu grammar rule.

Keterbatasan dari Recursive Descent:

1. Memory overhead: Recursive calls menggunakan stack
2. Left recursion: Tidak dapat menangani left-recursive grammar
3. Ambiguity: Memerlukan grammar yang tidak ambigu, yaitu LL(1)

1.4. Parse Tree

Parse Tree adalah representasi graf hierarkis dari struktur sintaks program berdasarkan grammar bahasa pemrograman. Parse tree menunjukkan bagaimana token-token hasil lexical analysis dapat diturunkan dari start symbol melalui serangkaian grammar productions.

Struktur Parse Tree:

1. Root Node: Start symbol dari grammar (biasanya <program>)
2. Internal Nodes: Non-terminal symbols dari grammar
3. Leaf Nodes: Terminal symbols (tokens dari lexer)
4. Edges: Merepresentasikan grammar productions

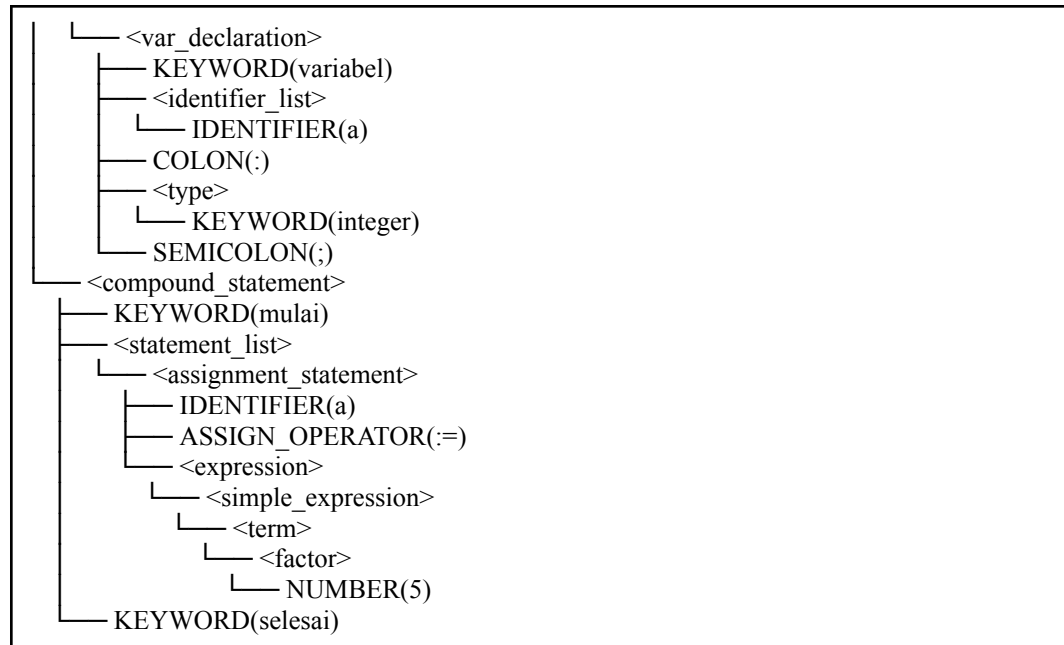
Parse tree dalam implementasi Pascal-S parser direpresentasikan sebagai list of tuples yang mencatat setiap node dengan level depth-nya, memungkinkan reconstruction struktur hierarkis untuk output formatting.

Contoh Parse Tree dari Pascal-S:

```

<program>
├── <program_header>
│   ├── KEYWORD(program)
│   ├── IDENTIFIER(Hello)
│   └── SEMICOLON(;)
└── <declaration_part>

```



Kegunaan Parse Tree:

1. Syntax validation: Membuktikan bahwa program memiliki struktur sintaks yang valid
2. Semantic analysis input: Digunakan sebagai input untuk tahap semantic analysis
3. Code generation: Memandu proses code generation dengan struktur yang jelas
4. Error reporting: Membantu dalam melokalisasi dan menjelaskan syntax errors
5. Program understanding: Memvisualisasikan struktur program untuk keperluan debugging atau dokumentasi

2. Perancangan dan Implementasi

2.1. Penyesuaian Rules pada dfa_rules.json

Pada milestone 2, terdapat penyesuaian dari keyword yang digunakan pada rulesnya. Berikut adalah perubahan yang ditambahkan dari milestone 1.

- a. Keyword Sebelum: "program", "var", "begin", "end", "if", "then", "else", "while", "do", "for", "to", "downto", "integer", "real", "boolean", "char", "array", "of", "procedure", "function", "const", "type", "div", "mod", "and", "or", "not", "writeln"
- b. Keyword Sesudah: "program", "variabel", "mulai", "selesai", "jika", "maka", "selain_itu", "selama", "lakukan", "untuk", "ke", "turun_ke", "integer", "real", "boolean", "char", "larik", "dari", "prosedur", "fungsi",

"konstanta", "tipe", "bagi", "mod", "dan", "atau", "tidak", "ke", "ulangi",
"rekaman", "kasus"

2.2. Penyesuaian Compiler dengan Tambahan Parser dan Pembentukan Parser Tree

Sebelumnya, compiler dibuat hanya untuk disesuaikan dengan algoritma lexical analysis, namun sekarang terdapat penyesuaian dalam bentuk line berikut:

```
parser = Parser(tokens)
try:
    list_tokens = parser.parse()
except ParserError as pe:
    print('Parser error:', pe, file=sys.stderr)
    sys.exit(4)
```

Token yang didapat dari hasil algoritma lexer terhadap suatu program Pascal-S akan dilanjutkan untuk diolah oleh parser. Disini, parser akan mengubah token tersebut dalam bentuk list yang disimpan dalam variabel `list_tokens`.

Lalu, list tersebut akan dibentuk menjadi sebuah tree melalui fungsi berikut:

```
def print_parse_tree(list_tokens):
    if not list_tokens:
        return

    active_levels = set()

    for idx, (token_str, depth) in enumerate(list_tokens):
        is_last = True
        for future_idx in range(idx + 1, len(list_tokens)):
            future_depth = list_tokens[future_idx][1]
            if future_depth < depth:
                break
            if future_depth == depth:
                is_last = False
                break

        prefix = ""
        for level in range(depth):
            if level in active_levels:
                prefix += " | "
            else:
                prefix += "  "

        if depth > 0:
            if is_last:
                prefix += " └── "
                if depth in active_levels:
                    active_levels.remove(depth)
            else:
                prefix += " ┌── "
```

```
active_levels.add(depth)

print(f'{prefix} {token_str}')
```

Fungsi tersebut akan mengolah list of tuple yang dihasilkan parser dengan menyesuaikan kedalaman sesuai input yang diterima. Suatu tuple akan berisi pendefinisian dari suatu node hasil dari fungsi untuk setiap node dan variabel *i* yang merupakan tingkat kedalaman suatu node. Kedalaman suatu program akan dimulai dengan node <program> dengan kedalaman yang akan dimulai dari 0. Penjelasan terkait sistem node dan kedalamannya akan dijelaskan pada sub-bab selanjutnya.

2.3. Penambahan Algoritma Syntax Analysis (Parser)

Parser diimplementasikan menggunakan pendekatan *recursive descent*, dimana parser akan menerima list of tokens dari lexer sebagai input, kemudian membangun parse tree berdasarkan grammar Pascal-S yang telah didefinisikan. Struktur dasar dari kelas Parser yang kami buat memiliki tiga atribut utama, yaitu tokens yang merupakan list of tokens hasil dari lexical analysis, position yang merupakan pointer untuk track posisi token yang sedang diproses, serta tree_list yang merupakan list of tuples dengan format (node_string, depth) yang merepresentasikan parse tree. Parser menggunakan dua global variabel yaitu sym (token yang sedang diproses) dan i (level kedalaman dalam parse tree).

Untuk memproses token, terdapat tiga fungsi utama yaitu next_token, accept, dan accept_identifier, yang dapat dilihat pada fungsi berikut.

```
def next_token(self):
    if self.position < len(self.tokens):
        token = self.tokens[self.position]
        self.position += 1
        return token
    else:
        return Token('EOF', 'EOF', -1, -1)

def accept(self, expected_type, expected_value):
    global sym, i
    i += 1
    if sym.type == expected_type and sym.value == expected_value:
        self.tree_list.append((str(sym), i))
        sym = self.next_token()
    else:
        raise ParserError(f'Expected {expected_value} of type {expected_type}, got {sym.value} of type {sym.type} at line {sym.line}, column {sym.column}')
        i -= 1

def accept_identifier(self):
    global sym, i
    i += 1
```

```

if sym.type == 'IDENTIFIER':
    self.tree_list.append((str(sym), i))
    sym = self.next_token()
else:
    raise ParserError(f'Expected IDENTIFIER, got {sym.value} at line {sym.line}, column {sym.column}')
    i -= 1

```

Fungsi `next_token()` berfungsi untuk mengambil token berikutnya dari sequence. Di sisi lain, fungsi `accept(expected_type, expected_value)` berfungsi untuk memvalidasi dan mengonsumsi token dengan tipe dan value spesifik. Sedangkan, `accept_identifier()` merupakan fungsi khusus untuk mengonsumsi token bertipe `identifier`.

Dalam pengimplementasian grammar rules, setiap non-terminal dalam grammar direpresentasikan sebagai method terpisah dengan pola yang konsisten, yaitu increment depth level ditambah 1, append non-terminal node ke `tree_list` dengan format ("`<non_terminal>`", `i`), proses grammar production dengan memanggil `accept()` untuk terminal atau method lain untuk non-terminal, dan decrement depth level `i` dikurangi 1.

Salah satu contoh implementasi adalah `<program_header>` yang mengikuti production `<program_header> ::= program <identifier>` ; dengan kode sebagai berikut:

```

def program_header(self):
    global sym,i
    i += 1
    self.tree_list.append(("<program_header>", i))
    self.accept('KEYWORD', 'program')
    self.accept_identifier()
    self.accept('SEMICOLON', ';')
    i -= 1

```

Untuk grammar rules dengan alternatif productions, parser menggunakan conditional branching berdasarkan lookahead tokennya. Seperti contoh, untuk `<declaration_part>`, menggunakan while loop untuk mengecek keyword 'konstanta', 'tipe' atau 'variabel'. Dapat dilihat pada contoh kode berikut:

```

def declaration_part(self):
    global sym,i
    i += 1
    self.tree_list.append(("<declaration_part>", i))
    while sym.type == 'KEYWORD' and sym.value in ('konstanta', 'tipe', 'variabel'):
        if sym.value == 'konstanta':
            self.constant_declaration()

```

```

elif sym.value == 'tipe':
    self.type_declaration()
elif sym.value == 'variabel':
    self.var_declaration()
while sym.type == 'KEYWORD' and (sym.value == 'prosedur' or sym.value == 'fungsi'):
    self.subprogram_declaration()
i -= 1

```

Untuk expression parsing, parser diimplementasikan dengan memisahkan berdasarkan operator precedence level (dari terendah ke tinggi). Sebagai contoh, <expression> yang berisikan relational operators (=, <, >, <=, >=), <simple_expression> yang berisikan additive operators (+, -, atau), <term> yang merupakan multiplicative operators (*, /, bagi, mod, dan), dan <factor> yang merupakan operands (identifier, number, string, parenthesized expression, tidak). Setiap level menggunakan while loop untuk menangani operator left-associative dengan benar. Contohnya dapat dilihat pada kode berikut ini.

```

def expression(self):
    global sym,i
    i += 1
    self.tree_list.append("<expression>", i)
    self.simple_expression()
    if sym.type == 'RELATIONAL_OPERATOR':
        self.accept('RELATIONAL_OPERATOR', sym.value)
        self.simple_expression()
    i -= 1

def simple_expression(self):
    global sym,i
    i += 1
    self.tree_list.append("<simple_expression>", i)
    if sym.type == 'ARITHMETIC_OPERATOR' and sym.value in ('+', '-'):
        self.accept('ARITHMETIC_OPERATOR', sym.value)
        self.term()
    while sym.type == 'ARITHMETIC_OPERATOR' and sym.value in ('+', '-'):
        self.additive_operator()
        self.term()
    i -= 1

def term(self):
    global sym,i
    i += 1
    self.tree_list.append("<term>", i)
    self.factor()
    while (sym.type == 'ARITHMETIC_OPERATOR' and sym.value in ('*', '/')) or (sym.type == 'KEYWORD' and sym.value in ('bagi', 'mod', 'dan')):
        self.multiplicative_operator()
        self.factor()
    i -= 1

```

```

def factor(self):
    global sym,i
    i += 1
    self.tree_list.append("<factor>", i))
    match sym.type:
        case 'IDENTIFIER':
            self.accept_identifer()
            if sym.type == 'LPARENTHESIS' and sym.value == '(':
                self.function_params()
        case 'NUMBER':
            self.accept('NUMBER',sym.value)
        case 'CHAR_LITERAL':
            self.accept('CHAR_LITERAL',sym.value)
        case 'STRING_LITERAL':
            self.accept('STRING_LITERAL',sym.value)
        case 'LPARENTHESIS':
            self.accept('LPARENTHESIS', '(')
            self.expression()
            self.accept('RPARENTHESIS',')')
        case 'KEYWORD':
            if sym.value == 'tidak':
                self.accept('KEYWORD','tidak')
                self.factor()
    i -= 1

```

Dalam menangani repetitions dan optional elements, kami mengimplementasikan repetitions (zero or more) menggunakan while loop dengan kondisi lookahead token, seperti pada contoh <identifier_list> yang melakukan pengecekan pada COMMA.

```

def identifier_list(self):
    global sym,i
    i += 1
    self.tree_list.append("<identifier_list>", i))
    self.accept_identifer()
    while sym.type == 'COMMA' and sym.value == ',':
        self.accept('COMMA', ',')
        self.accept_identifer()
    i -= 1

```

Sedangkan untuk optional element, kami menggunakan conditional check sebelum parsing, seperti contoh pada <if_statement> yang melakukan pengecekan keberadaan keyword 'selain_itu'.

```

def if_statement(self):
    global sym,i
    i += 1
    self.tree_list.append("<if_statement>", i))
    self.accept('KEYWORD', 'jika')

```

```
self.expression()
self.accept('KEYWORD', 'maka')
self.statement()
if sym.type == 'KEYWORD' and sym.value == 'selain_itu':
    self.accept('KEYWORD', 'selain_itu')
    self.statement()
i -= 1
```

Seluruh fungsi ini akan dimulai dengan method `parse()` yang menjadi entry point yang memulai parsing dari root grammar, dengan kode sebagai berikut.

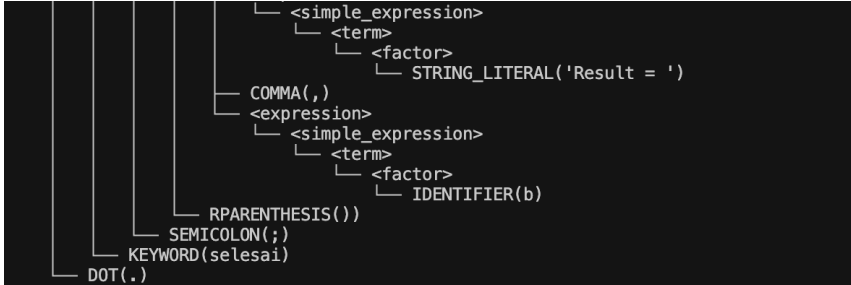
```
def parse(self):
    global sym, i
    # berarti mulai dari program
    sym = self.next_token()
    self.tree_list.append("<program>", i)
    self.program_header()
    self.declaration_part()
    self.compound_statement()
    self.accept('DOT', '.')
    return self.tree_list
```

Return value berupa `tree_list` nantinya akan divisualisasikan oleh fungsi `print_parse_tree()`. Pada dasarnya, fungsi ini akan mengiterasi setiap tuple dan membangun tree structure dengan menggunakan unicode box drawing untuk membuat visualisasi tree structure-nya.

3. Pengujian

3.1. Test Case Benar

Input	<pre>program Hello; variabel a, b: integer; mulai a := 5; b := a + 10; writeln('Result = ', b); selesai.</pre>
Output	<pre>saktibimasena@SAKTIs-MacBook-Pro NTT-Tubes-IF2224 % python3 -m src.compiler test/mi as <program> ├── <program_header> │ ├── KEYWORD(program) │ ├── IDENTIFIER(Hello) │ └── SEMICOLON(;) ├── <declaration_part> │ └── <var_declaration> │ ├── KEYWORD(variabel) │ ├── <identifier_list> │ │ ├── IDENTIFIER(a) │ │ ├── COMMA(,) │ │ └── IDENTIFIER(b) │ ├── COLON(:) │ ├── <type> │ │ └── KEYWORD(integer) │ └── SEMICOLON(;) ├── <compound_statement> │ ├── KEYWORD(mulai) │ └── <statement_list> │ ├── <assignment_statement> │ │ ├── IDENTIFIER(a) │ │ ├── ASSIGN_OPERATOR(:=) │ │ └── <expression> │ │ └── <simple_expression> │ │ └── <term> │ │ └── <factor> │ │ └── NUMBER(5) │ ├── SEMICOLON(;) │ ├── <assignment_statement> │ │ ├── IDENTIFIER(b) │ │ ├── ASSIGN_OPERATOR(:=) │ │ └── <expression> │ │ └── <simple_expression> │ │ ├── <term> │ │ │ └── <factor> │ │ │ └── IDENTIFIER(a) │ │ ├── <additive_operator> │ │ │ └── ARITHMETIC_OPERATOR(+) │ │ └── <term> │ │ └── <factor> │ │ └── NUMBER(10) │ ├── SEMICOLON(;) │ └── <procedure_function_call> │ ├── IDENTIFIER(writeln) │ ├── LPARENTHESIS(() │ ├── <parameter_list> │ │ └── <expression></pre>

	 <pre> graph TD Root[] --- DOT[DOT(.)] Root --- KEYWORD[KEYWORD(selesai)] Root --- SEMICOLON[SEMICOLON(;)] Root --- RPARENTHESIS[RPARENTHESIS())] Root --- COMMA[COMMA(,)] Root --- EXPRESSION[expression] EXPRESSION --- SIMPLE_EXPRESSION1[simple_expression] SIMPLE_EXPRESSION1 --- TERM1[term] TERM1 --- FACTOR1[factor] FACTOR1 --- STRING_LITERAL[STRING_LITERAL('Result = ')] EXPRESSION --- SIMPLE_EXPRESSION2[simple_expression] SIMPLE_EXPRESSION2 --- TERM2[term] TERM2 --- FACTOR2[factor] FACTOR2 --- IDENTIFIER[IDENTIFIER(b)] </pre>
Keterangan	Test case valid dan hasil output sesuai

3.2. Error, Kurang Semicolon

Input	<pre> program ErrorSemicolon; variabel x, y : integer; z : real mulai x := 10; y := 20 z := x + y writeln('Hasil: ', z) selesai. </pre>
Output	<pre> saktibimasena@SAKTIIs-MacBook-Pro NTT-Tubes-IF2224 % python3 -m src.compiler test/milestone2/test2.p as Parser error: Expected ; of type SEMICOLON, got mulai of type KEYWORD at line 7, column 1 </pre>
Keterangan	y := 20 kurang semicolon (;), parser berhasil menangkapnya

3.3. Error, Missing Syntax

Input	<pre> program ErrorMissingSyntax; variabel x, y, z : integer; hasil : real; prosedur hitung(a : integer); mulai x := a * 2 </pre>
-------	---

	<pre> selesai; mulai x := 10; y := 20; jika x > y z := x selain_itu z := y; selama z > 0 z := z - 1; writeln('X = ', x); writeln('Y = ', y); writeln('Z = ', z); selesai. </pre>
Output	<pre> muhammadfithrarizki@Muhammad-Fithras-Macbook-Pro NTT-Tubes-IF2224 % python3 -m src.compiler test/mi. Parser error: Expected maka of type KEYWORD, got z of type IDENTIFIER at line 17, column 9 </pre>
Keterangan	Blok <pre> jika x > y z := x selain_itu z := y; </pre> Kurang maka, jika x > y (maka) Parser berhasil menangkapnya.

3.4. Error, Tutup Kurung Tidak Sesuai

Input	<pre> program ErrorParenthesis; variabel x, y, z : integer; mulai x := 5; y := 10; writeln('X = ', x, ', Y = ', y); </pre>
-------	--

	selesai.
Output	<pre>muhammadfithrarizki@Muhammad-Fithras-Macbook-Pro NTT-Tubes-IF2224 % python3 -m src.compiler test/mi Parser error: Expected) of type RPARENTHESIS, got] of type RBRACKET at line 9, column 35</pre>
Keterangan	Tutup kurung mismatch di <code>writeln('X = ', x, ', Y = ', y];</code> Harusnya <code>writeln('X = ', x, ', Y = ', y);</code> Parser berhasil menangkapnya

3.5. Error, Missing Structure

Input	<pre>program ErrorStructure; variabel x : integer; arr : larik[1..10] dari integer; variabel a, b : ; fungsi calculate(n : integer) mulai calculate := n * 2 selesai; mulai writeln('Test', x); x := 5 + 10; selesai.</pre>
Output	<pre>muhammadfithrarizki@Muhammad-Fithras-Macbook-Pro NTT-Tubes-IF2224 % python3 -m src.compiler test/mi Parser error: Expected IDENTIFIER, got ; at line 8, column 12</pre>
Keterangan	Kurang nilai yang didefinisikan pada blok variabel <code>a, b : ;</code> Parser berhasil menangkapnya

4. Kesimpulan dan Saran

4.1. Kesimpulan

Implementasi syntax analysis untuk compiler Pascal-S pada milestone 2 telah berhasil diselesaikan dengan menggunakan algoritma recursive descent parsing. Berdasarkan hasil pengujian dan implementasi yang dilakukan, dapat ditarik beberapa kesimpulan sebagai berikut:

1. **Keberhasilan Implementasi Parser:** Parser berhasil diimplementasikan dengan pendekatan top-down recursive descent yang mengikuti struktur grammar Pascal-S secara langsung. Setiap non-terminal dalam grammar direpresentasikan sebagai method terpisah dalam class Parser, yang memudahkan dalam pemeliharaan dan debugging kode. Parser mampu memproses token sequence yang dihasilkan oleh lexer dari milestone 1 dan menghasilkan parse tree yang merepresentasikan struktur hierarkis program Pascal-S.
2. **Parse Tree Generation dan Error Handling:** Parser berhasil menghasilkan parse tree dalam format yang terstruktur dengan visualisasi menggunakan unicode box drawing characters. System error handling yang diimplementasikan mampu mendeteksi berbagai jenis syntax error dengan pesan yang informatif, termasuk missing semicolon, syntax yang hilang, dan parentheses mismatch. Parser terintegrasi dengan baik dengan lexer dari milestone 1 dan berhasil mengadaptasi keyword bahasa Indonesia sesuai spesifikasi.

4.2. Saran

Berdasarkan hasil implementasi dan pengujian yang telah dilakukan, terdapat beberapa aspek yang dapat ditingkatkan untuk pengembangan lebih lanjut:

1. **Peningkatan Error Recovery dan Optimisasi:** Saat ini parser berhenti pada error pertama yang ditemukan. Untuk meningkatkan user experience, dapat diimplementasikan error recovery mechanism yang memungkinkan parser melanjutkan proses parsing setelah menemukan error. Selain itu, untuk tahap semantic analysis selanjutnya, dapat dipertimbangkan implementasi Abstract Syntax Tree (AST) yang lebih efisien dalam hal memory usage dan processing speed.
2. **Peningkatan Kualitas dan Coverage Testing:** Pesan error dapat ditingkatkan dengan memberikan suggestion atau hint kepada user tentang kemungkinan perbaikan yang dapat dilakukan. Coverage testing juga dapat diperluas untuk mencakup lebih banyak edge cases dan kombinasi syntax yang kompleks, serta optimasi performa untuk program Pascal-S yang lebih besar dan kompleks.

5. Lampiran

Link Release Repository Github : <https://github.com/fithrarzk/NTT-Tubes-IF2224>

Aturan Grammar

Notasi EBNF yang digunakan:

+ : Konkatenasi

| : Pilihan (OR)

{ ... } : Nol atau lebih pengulangan

[...] : Nol atau satu (opsional)

BOLD CAPS : Simbol Terminal (Token)

<nama_node> : Simbol Non-Terminal

No	Nama Node	Hasil Produksi
1	<program>	<program-header> + <declaration-part> + <compound-statement> + DOT(.)
2	<program-header>	KEYWORD(program) + IDENTIFIER + SEMICOLON(;)
3	<declaration-part>	{<const-declaration>} + {<type-declaration>} + {<var-declaration>} + {<subprogram-declaration>}
4	<const-declaration> >	KEYWORD(konstanta) + IDENTIFIER + RELATIONAL_OPERATOR(=) + value + SEMICOLON(;) + { IDENTIFIER + RELATIONAL_OPERATOR(=) + value + SEMICOLON(;)}
5	<type-declaration>	KEYWORD(tipe) + IDENTIFIER + RELATIONAL_OPERATOR(=) <type-definition> + SEMICOLON(;) + { IDENTIFIER + RELATIONAL_OPERATOR(=) <type-definition> + SEMICOLON(;)}
6	<var-declaration>	KEYWORD(variabel) + <identifier-list> + COLON(:) + <type> + SEMICOLON(;) + {<identifier-list> + COLON(:) + <type> + SEMICOLON(;)}
7	<identifier-list>	IDENTIFIER + { COMMA(,) + IDENTIFIER }
8	<type>	KEYWORD(integer) KEYWORD(char) KEYWORD(real) KEYWORD(boolean) <array-type> IDENTIFIER

9	<array-type>	KEYWORD(larik) + LBRACKET() + <range> + RBRACKET() + KEYWORD(dari) + <type>
10	<range>	<expression> + RANGE_OPERATOR(..) + <expression>
11	<subprogram-declaration>	<procedure-declaration> <function-declaration>
12	<procedure-declaration>	KEYWORD(prosedur) + IDENTIFIER + [formal-parameter-list] + SEMICOLON(;) + <block> + SEMICOLON(;)
13	<function-declaration>	KEYWORD(fungsi) + IDENTIFIER + [formal-parameter-list] + COLON(:) + <type> + SEMICOLON(;) + <block> + SEMICOLON(;)
14	<parameter-group>	<identifier-list> + COLON(:) + <type>
15	<formal-parameter-list>	LPARENTHESIS() + <parameter-group> { SEMICOLON(;) + <parameter-group> } + RPARENTHESIS()
16	<compound-statement>	KEYWORD(mulai) + <statement-list> + KEYWORD(selesai)
17	<statement-list>	(<assignment-statement> <if-statement> <compound-statement> <while-statement> <for-statement> <procedure_function_call>) + {<identifier-list> + COLON(:) + <type> + SEMICOLON(;) + {(<assignment-statement> <if-statement> <compound-statement> <while-statement> <for-statement> <procedure_function_call>)}}
18	<assignment-statement>	IDENTIFIER + ASSIGN_OPERATOR(:=) + <expression>
19	<if-statement>	KEYWORD(jika) + <expression> + KEYWORD(maka) + (<assignment-statement> <if-statement> <compound-statement> <while-statement> <for-statement> <procedure_function_call>) + [KEYWORD(selain-itu) + (<assignment-statement> <if-statement> <compound-statement> <while-statement> <for-statement> <procedure_function_call>)]

20	<while-statement>	KEYWORD(selama) + <expression> + KEYWORD(lakukan) + (<assignment-statement> <if-statement> <compound-statement> <while-statement> <for-statement> <procedure_function_call>)
21	<for-statement>	KEYWORD(untuk) + IDENTIFIER + ASSIGN_OPERATOR(:=) + <expression> + (KEYWORD(ke) KEYWORD(turun-ke)) + <expression> + KEYWORD(lakukan) + (<assignment-statement> <if-statement> <compound-statement> <while-statement> <for-statement> <procedure_function_call>)
22	<procedure_function_call>	IDENTIFIER + (LPARENTHESIS() + <parameter-list> + RPARENTHESIS())
23	<parameter-list>	<expression> + { COMMA(,) + <expression>}
24	<expression>	<simple-expression> + [RELATIONAL_OPERATOR(=) + <simple-expression>]
25	<simple-expression>	[ARITHMETIC_OPERATOR(+) ARITHMETIC_OPERATOR(-)] + <term> + {<additive-operator> + <term>}
26	<term>	<factor> + {<multiplicative-operator> + <factor>}
27	<factor>	IDENTIFIER NUMBER CHAR_LITERAL STRING_LITERAL LPARENTHESIS() + <expression> + RPARENTHESIS() KEYWORD(tidak) + <factor> <procedure-function-call>
28	<relational-operator>	RELATIONAL_OPERATOR(=) RELATIONAL_OPERATOR(< >) RELATIONAL_OPERATOR(<) RELATIONAL_OPERATOR(<=) RELATIONAL_OPERATOR(>) RELATIONAL_OPERATOR(>=)
29	<additive-operator>	ARITHMETIC_OPERATOR(+) ARITHMETIC_OPERATOR(-)

		KEYWORD(atau)
30	<multiplicative-operator>	ARITHMETIC_OPERATOR(*) ARITHMETIC_OPERATOR(/) KEYWORD(bagi) KEYWORD(mod) KEYWORD(dan)
31	<block>	<declaration-part> + <compound-statement>
32	<type-definition>	KEYWORD(larik) + LBRACKET([) + <range> + RBRACKET(]) + KEYWORD(dari) + <type> <type>

Pembagian Tugas

NIM	Nama	Pembagian Tugas	Presentasi Kontribusi
13523047	Indah Novita Tangdililing	• Node 1-6 • Laporan	19%
13523049	Muhammad Fithra Rizki	• Node 7-12 • Laporan	19%
13523053	Sakti Bimasena	• Node 13-18 • Parse Tree • Laporan	24%
13523055	Muhammad Timur Kanigara	• Node 19-24 • Laporan	19%
13523113	Kefas Kurnia Jonathan	• Node 25-30 • Laporan	19%

6. Referensi

Judhi S., Parse Tree and Conversion IF2124 Teori Bahasa Formal Otomata. (Diakses via edunex).

Rila Mandala, Konsep dan Notasi Bahasa. (Diakses via edunex).

Hopcroft et al, Introduction to Automata Theory, Languages, and Computation.